

Software Requirements Specification

Sep 8th, 2026

1. Register for an Account

- Actor: Driver
- Precondition: User has downloaded the app, has a valid email
- Main flow: User enters email/phone → creates password → verifies via code → profile created
- Alt flow: If email already exists, system prompts user to log in instead
- Postcondition: Account created, user logged in

2. Reserve a Parking Space

- Actor: Driver
- Precondition: User is logged in, has selected a garage
- Main flow: User selects available spot → picks time window → system shows total price → user confirms → payment processed → confirmation generated
- Alt flow: If no spots available, system suggests nearby garages
- Postcondition: Spot held for user, confirmation sent

3. View Real-Time Parking Availability

- Actor: Driver
- Precondition: Location services enabled
- Main flow: User opens map → system pulls live occupancy data from nearby garages → available spots displayed
- Alt flow: If data feed is delayed, system flags listing as "last updated X min ago"
- Postcondition: User sees accurate, timestamped availability

4. Cancel a Reservation

- Actor: Driver
- Precondition: User has an active reservation
- Main flow: User selects reservation → chooses cancel → system checks cancellation window → refund issued if eligible
- Alt flow: If outside refund window, system shows partial or no refund and asks for confirmation
- Postcondition: Reservation canceled, spot released back to inventory

5. Extend an Active Parking Session

- Actor: Driver
- Precondition: User has a currently active session

- Main flow: User selects "extend time" → system checks spot availability for extended window → payment processed → session updated
- Alt flow: If spot is booked by someone else after user's window, system offers nearby alternatives
- Postcondition: Session time updated, new expiration set

6. Process a Digital Payment

- Actor: Driver
- Precondition: User has a valid payment method on file
- Main flow: User confirms reservation → system charges payment method → receipt generated
- Alt flow: If payment fails, system prompts user to retry or use another card
- Postcondition: Payment completed, funds routed to operator account

7. View Reservation History and Receipts

- Actor: Driver
- Precondition: User is logged in and has at least one past reservation
- Main flow: User navigates to history tab → system displays past reservations with receipts → user selects one to view detail
- Alt flow: If no history exists, system displays empty state message
- Postcondition: User views/downloads receipt

8. Receive Expiration Alert

- Actor: System (triggers notification to Driver)
- Precondition: User has an active reservation nearing end time
- Main flow: System detects session ending in 10 minutes → sends push notification → offers option to extend
- Alt flow: If user has notifications disabled, no alert sent, session simply expires
- Postcondition: User notified, can act before session ends

9. Get Navigation to Reserved Spot

- Actor: Driver
- Precondition: User has confirmed reservation
- Main flow: User taps "navigate" → system pulls route via integrated mapping service → turn-by-turn directions displayed
- Alt flow: If mapping service is unavailable, system displays static address and garage entrance info
- Postcondition: User arrives at garage with directions provided

10. Monitor Occupancy Dashboard

- Actor: Parking Operator

- Precondition: Operator is logged into admin dashboard
- Main flow: Operator opens dashboard → system displays real-time occupancy across all garage levels → operator reviews trends
- Alt flow: If sensor data from a level is offline, dashboard flags that section as "data unavailable"
- Postcondition: Operator has current occupancy snapshot

11. Adjust Garage Pricing

- Actor: Parking Operator
- Precondition: Operator has admin access to a specific garage
- Main flow: Operator selects garage → updates hourly/dynamic pricing rules → system saves and applies changes going forward
- Alt flow: If new pricing conflicts with an active reservation's locked-in rate, system honors the original rate for that reservation
- Postcondition: Updated pricing live for new reservations

12. Generate Occupancy and Revenue Report

- Actor: Parking Operator
- Precondition: Operator account has reporting access
- Main flow: Operator selects date range → system compiles occupancy and revenue data → report generated and exportable
- Alt flow: If data range spans an outage period, report flags gaps in data
- Postcondition: Operator downloads/views report

13. Manage User Accounts (Admin)

- Actor: System Administrator
- Precondition: Admin is logged into backend system
- Main flow: Admin searches for user account → views account status → can suspend, reinstate, or edit account details
- Alt flow: If account has pending disputes, system warns admin before allowing suspension
- Postcondition: Account status updated

14. Resolve a Payment Dispute

- Actor: Finance/Billing Personnel
- Precondition: A dispute has been filed by a driver
- Main flow: Billing staff reviews transaction details → contacts driver and/or operator if needed → issues refund, partial refund, or denies dispute → resolution logged
- Alt flow: If dispute involves suspected fraud, case is escalated to security review
- Postcondition: Dispute closed, resolution recorded in system

15. Sync with Third-Party Mapping Service

- Actor: System (Backend/Mapping Integration)
- Precondition: Mapping API credentials are valid and active
- Main flow: System sends periodic requests to mapping service → receives updated map data and traffic conditions → integrates into driver-facing map view
- Alt flow: If API call fails or times out, system falls back to last cached map data and flags it as potentially outdated
- Postcondition: Map data refreshed or fallback data used without app crash

AI was used to simulate actors needing to complete an action on the app when I ran out of ideas on how to use the app.